



STANDALONE GUITAR TUNER

Guitar Tuner

User Manual & Technical Documentation

Built with JUCE 8.0.15 · C++17 · Windows / ASIO

github.com/radudeacu/guitar-tuner-juce

Contents

This document is in two halves. Part I is for playing the instrument; Part II is for working on the code. Neither assumes you have read the other.

Part I — User Manual

1	About this tuner	Manual
2	Requirements and first launch	Manual
3	Reading the display	Manual
4	Tuning your guitar	Manual
5	Tunings and presets	Manual
6	The reference tone	Manual
7	Options	Manual
8	What the app remembers	Manual
9	Troubleshooting	Manual

Part II — Technical Documentation

10	Architecture at a glance	Code
11	Building the project	Code
12	The threading model	Code
13	The audio signal path	Code
14	Pitch detection — the YIN algorithm	Code
15	The plucked string synthesiser	Code
16	Reference tone playback	Code
17	The tuning model	Code
18	The user interface layer	Code
19	Settings persistence	Code
20	Tests and developer tools	Code
21	File reference	Code
22	Known limitations and deferred scope	Code

PART I

User Manual

Everything you need to tune a guitar with this application — what the display is telling you, how to pick a tuning, and how to use the reference tone to tune by ear.

1 · About this tuner

A standalone chromatic-style tuner for six-string guitar, built for Windows and designed around a real audio interface rather than a laptop microphone. It listens to one note at a time, works out its pitch, and shows how far that pitch sits from the string you are aiming for.

What it does

Feature	Summary
Live pitch detection	Detects a single played note from roughly 70 Hz to 1400 Hz, covering every fretted position on a standard guitar.
Seven tunings	Standard, Drop D, Open D, Open G, Open E, DADGAD and Half Step Down.
Reference tone	Plays any string's target pitch as a plucked guitar note, so you can tune by ear as well as by eye.
Adjustable concert pitch	A4 anywhere from 415 Hz to 466 Hz, for matching a piano or playing at historical pitch.
ASIO support	Low-latency input from any ASIO interface, with no separate SDK download needed.
Remembers your setup	Your interface, tuning and concert pitch are all restored next time you open the app.

What it deliberately does not do

The tuner listens to **one note at a time**. Strumming a chord will not give a meaningful reading — pluck a single string and let it ring. It is also built for six-string guitar specifically: bass guitar sits below its detection range, and there is no strobe display or plugin version.

2 · Requirements and first launch

What you need

Item	Detail
Operating system	Windows 10 or later, 64-bit.
Audio interface	Any ASIO-compatible interface. Developed and tested against a Focusrite Scarlett 2i2. A built-in microphone will work but gives noticeably less reliable readings.
Guitar	Six-string guitar, electric or acoustic, connected through your interface.

Setting up your audio input

On first launch the app picks a sensible default device, but you should confirm it is listening to your guitar:

- 1 Click **Options** in the top-left corner.
- 2 Set **Audio device type** to **ASIO** if your interface offers it. ASIO gives the lowest latency and the most stable readings.
- 3 Choose your interface under **Input** — for a Scarlett this appears as *Focusrite USB ASIO*.
- 4 Tick the input channel your guitar is plugged into.

5 Close the dialog. Play a string; the display should respond immediately.

Set your input gain properly

The tuner ignores anything below a quiet threshold so that room noise is not mistaken for a note. If readings look erratic or nothing registers at all, the usual cause is interface gain set too low. Turn the gain up until a plucked string registers clearly on your interface's own meter.

3 · Reading the display

The main screen is one large card. Everything on it describes a single played note.

Element	Where	What it tells you
Arc meter	Top	A needle sweeping across a 100-cent range. Dead centre is perfectly in tune; left is flat, right is sharp. The green band at the top marks the in-tune zone.
Note name	Middle, large	The string you are closest to, with its octave in smaller type beside it. A2 means the open A string on a standard-tuned guitar.
Direction hint	Below the note	TUNE UP or TUNE DOWN , appearing only when you are more than 5 cents out.
Cents offset	Below the hint	Exactly how far off you are. A cent is one hundredth of a semitone. Positive is sharp, negative is flat.
Frequency	Bottom	The raw detected pitch in hertz, for when you want the actual number.

The colour code

Colour is the fastest read on the screen. The needle, the note name and the cents figure all change together:

Colour	Range	Meaning
Green	Within 5 cents	In tune. Stop adjusting.
Amber	5 to 20 cents	Close. Small adjustments from here.
Red	More than 20 cents	Noticeably out. Keep turning.
Grey	—	No signal. The tuner cannot hear a note.

Why the needle glides instead of jumping

A plucked string wavers in pitch as it decays, especially in the first moment after the attack. The needle deliberately eases toward each new reading rather than snapping to it, so that natural waver reads as smooth movement instead of jitter. When the note dies away the needle returns to centre and the display greys out.

4 · Tuning your guitar

- 1 Pick your tuning from the dropdown in the top-right corner. It starts on **Standard**.
- 2 Pluck one string cleanly and let it ring. Do not strum — the tuner reads one note at a time.
- 3 Read the note name. It shows which string the tuner thinks you are aiming for.
- 4 Watch the needle and the colour. Red or amber to the left means flat; turn the peg to raise the pitch. To the right means sharp; lower it.
- 5 Adjust until the needle sits in the green band and the reading settles within a few cents of zero.
- 6 Repeat for each string, then check them all again — tuning one string changes the neck tension slightly and pulls the others with it.

Tune up to the note, not down to it

Guitarists traditionally approach the target pitch from below. Arriving upward leaves the string pulling against the tuning peg and the nut, which holds far better than arriving downward. If you overshoot and go sharp, drop clearly below the note and come back up.

If the note name looks wrong

The tuner matches whatever it hears to the *nearest* string in the current tuning. If a string is badly out — more than roughly half a semitone — it may land closer to a neighbouring string and show that one's name instead. Get roughly into the right region by ear first, or use the reference tone described next, and the reading will settle onto the correct string.

5 • Tunings and presets

Choose a tuning from the dropdown in the top-right. The reference tone buttons along the bottom relabel themselves to match, so they always show the six notes you are actually aiming for.

Preset	Strings, low to high	Typical use
Standard	E2 A2 D3 G3 B3 E4	The default tuning for almost all guitar music.
Drop D	D2 A2 D3 G3 B3 E4	Low string dropped a tone. Rock and metal; makes power chords playable with one finger.
Open D	D2 A2 D3 F#3 A3 D4	Open strings sound a D major chord. Slide and folk fingerstyle.
Open G	D2 G2 D3 G3 B3 D4	Open strings sound a G major chord. Blues and slide playing.
Open E	E2 B2 E3 G#3 B3 E4	Open strings sound an E major chord. Blues slide, at higher string tension.
DADGAD	D2 A2 D3 G3 A3 D4	Modal and ambiguous rather than major or minor. Celtic and fingerstyle.
Half Step Down	D#2 G#2 C#3 F#3 A#3 D#4	Everything lowered one semitone. Eases vocal keys and slackens string tension.

Sharps, not flats

Accidentals are always written as sharps. Half Step Down therefore reads **D#2 G#2 C#3 F#3 A#3 D#4**, where guitarists more often write *Eb Ab Db Gb Bb Eb*. These are the same pitches under different names; the app uses one convention consistently across every tuning.

6 · The reference tone

Along the bottom of the window sits a row of six buttons, one per string. Each plays that string's target pitch as a plucked guitar note, letting you tune by ear alongside the display.

Playing a note

Click any string button and it sounds once, ringing for a few seconds before fading to silence. Click it again to hear it again. Click a different string and it switches immediately. The button you last played stays highlighted so you can see which reference is selected.

Loop

The **Loop** toggle above the buttons keeps the note repeating. Rather than replaying on a fixed timer, each repeat begins the moment the previous note has fully decayed — so the rhythm follows the string's own ring, with no awkward overlap or silence. Switch Loop off and the note currently sounding simply finishes and stops.

Tuning by ear with beats

Play the reference tone and your string together. When they are close but not identical you will hear a slow pulsing — the two pitches beating against each other. The further apart they are, the faster it pulses. Adjust until the pulsing slows to a stop, and the strings are in unison. This is more sensitive than any visual display and is worth learning.

It will not confuse the tuner

The reference tone is routed only to your outputs and never reaches the pitch detector's input. You can leave a tone looping while you tune and the needle will keep tracking your guitar, not the tone. Your guitar signal is likewise not passed through to the speakers; if you want to hear yourself, monitor through your interface as usual.

7 · Options

Click **Options** in the top-left. These are the settings you configure once and then leave alone — which is why the tuning selector lives on the main screen instead.

Reference pitch (A4)

Sets the concert pitch the whole tuning is derived from. The default of 440 Hz is the modern standard. The slider covers 415 Hz to 466 Hz in half-hertz steps; double-click it to snap back to 440. Every string's target moves with it, so at 432 Hz the entire guitar is tuned proportionally lower.

You would change this to match a piano tuned away from concert pitch, to play with period instruments at baroque pitch, or because your ensemble has agreed on something other than 440.

Audio device settings

The standard device panel: driver type, input device, active channels, sample rate and buffer size. A smaller buffer means lower latency and a slightly more immediate display, at the cost of more CPU. If you hear crackling or the app struggles, increase it.

8 · What the app remembers

Close the app and reopen it and you should be able to start playing immediately, with no setup. Three things are stored:

Setting	Notes
Audio device	Driver type, device, sample rate and buffer size. Saved whenever you change it, not only on exit, so an unexpected shutdown cannot lose it.
Selected tuning	Stored by name, so future versions can add or reorder presets without your choice being misread.
Reference pitch	Your A4 setting.

Settings live in %APPDATA%\GuitarTuner\Guitar Tuner.settings, a small XML file. Deleting it resets the app to defaults.

9 · Troubleshooting

Symptom	Likely cause and fix
Display always says <i>no signal</i>	The tuner is not hearing you. Check the correct input device and channel are selected in Options, that your guitar is plugged in, and that interface gain is high enough.
Readings jump around erratically	Usually a weak or noisy input. Raise interface gain. If you are using a laptop microphone rather than an interface, expect this — room noise and mains hum can be mistaken for a note.
No ASIO option in the device list	Your interface's ASIO driver is not installed. Install the manufacturer's driver — for Focusrite this ships with Focusrite Control.
Note name is not the string I am playing	That string is far enough out of tune to sit closer to a neighbour. Get roughly into range by ear or with the reference tone, then fine-tune.
Reference tone is silent	No output device is selected. Open Options and confirm your interface's outputs are chosen and your monitoring is turned up.
Crackling or dropouts	Buffer size too small for your machine. Increase it in Options.
Needle moves but never turns green	You may be between two notes entirely, or in the wrong tuning — check the dropdown matches how your guitar is actually tuned.

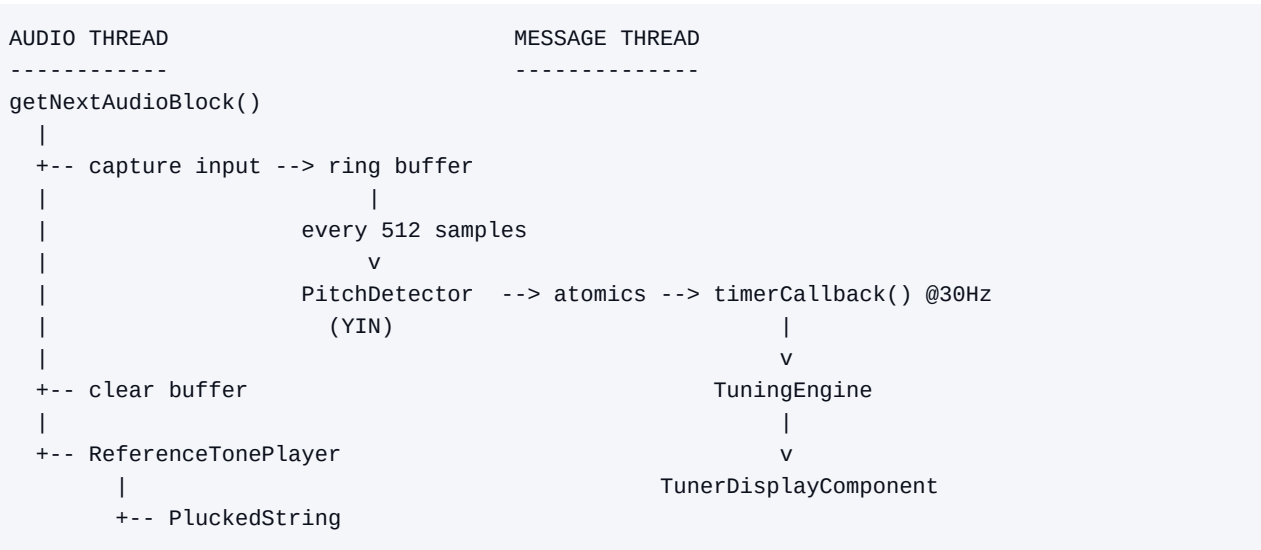
PART II

Technical Documentation

How the application is put together — its threading rules, the DSP behind pitch detection and tone synthesis, and a reference for every class in the codebase.

10 · Architecture at a glance

The codebase separates three concerns strictly: DSP that knows nothing about the interface, a tuning model that knows nothing about audio, and a UI layer that only ever reads results. Nothing in the DSP layer includes a JUCE GUI header.



The audio thread never touches a Component, and the message thread never touches the detector. The only things crossing between them are three scalar atomics and, in the other direction, the tone player's control flags.

Layers

Layer	Classes	Rule it obeys
DSP	PitchDetector, PluckedString	No JUCE GUI types. Testable standalone. No allocation after prepare().
Audio plumbing	ReferenceTonePlayer, MainComponent audio callback	Real-time safe. Cross-thread state via atomics only.
Model	Tuning, TuningPresets, TuningEngine, PitchMath	Pure computation. No audio, no UI, no threading concerns.
UI	TunerDisplayComponent, NeedleMeterComponent, ReferenceToneBar, OptionsPanel, Theme, GlassStyle, TunerLookAndFeel	Message thread only. Reads results, never computes them.
Persistence	TunerSettings	Message thread only. Wraps juce::ApplicationProperties.

Why this shape

The project is standalone today but AAX was named as a future target. Keeping the DSP free of any dependency on AudioAppComponent means the audio layer could be lifted into a PluginProcessor with the detector and synthesiser moving across untouched.

11 · Building the project

Component	Version / detail
Framework	JUCE 8.0.15, vendored as a git submodule at ./JUCE
Language	C++17
Build system	CMake 3.22+ using JUCE's official CMake API
Compiler	Visual Studio 2022 (MSVC 19.4x), 64-bit
JUCE modules	juce_audio_utils, juce_dsp, juce_gui_extra (which pull in audio_basics, audio_devices, gui_basics, graphics, events, core, data_structures)

```
git clone --recurse-submodules https://github.com/radudeacu/guitar-tuner-juce.git
cd guitar-tuner-juce
cmake -B build -G "Visual Studio 17 2022" -A x64
cmake --build build --config Debug
```

If you cloned without `--recurse-submodules`, run `git submodule update --init` first.

Build targets

Target	Purpose
GuitarTuner	The application itself.
TunerTests	Console test runner. No third-party test framework; exits non-zero on failure.
DesignPreview	Developer tool. Renders the tuner display in several tuning states, plus the options panel, to PNG files — so UI work can be reviewed without a guitar connected.

ASIO

ASIO is enabled unconditionally through the `JUCE_ASIO=1` compile definition. Since JUCE 8.0.11 the framework bundles the small set of Steinberg headers it needs, so no separate SDK download or licence acceptance is required. Earlier JUCE versions did require the SDK on the include path; if you downgrade the submodule below 8.0.11 the build will fail on a missing `iasiodrv.h`.

12 · The threading model

Two threads, with a strict contract between them. Getting this wrong in audio code produces glitches that are hard to reproduce and harder to diagnose, so the rules are worth stating plainly.

Rules the audio thread obeys

Rule	How it is honoured
No allocation	Ring buffer, analysis scratch buffer and all DSP state are sized in <code>prepareToPlay()</code> . The render path only writes into existing storage.
No locks	Nothing on the audio path takes a mutex. Cross-thread values are <code>std::atomic</code> with relaxed ordering, which is sufficient for independent scalars.
No UI access	The audio thread never calls into a Component. It publishes scalars and stops there.
No blocking calls	No file, network or logging calls on the render path.

What crosses the boundary

```
// audio thread writes, message thread reads
std::atomic<float> latestFrequencyHz;
std::atomic<float> latestConfidence;
std::atomic<bool>  signalPresent;

// message thread writes, audio thread reads and consumes
std::atomic<float> requestedFrequencyHz;
std::atomic<bool>  pluckRequested;    // consumed with exchange(false)
std::atomic<bool>  looping;
```

No lock-free FIFO is needed. Sample data never crosses threads — capture and analysis both happen on the audio thread — so the only shared state is a handful of scalars, and plain atomics are the right tool. A queue here would be machinery without a purpose.

Why TuningEngine needs no synchronisation

TuningEngine is mutated from UI callbacks and read from the 30 Hz timer. Both run on the message thread, so its state is never touched concurrently despite sitting one step away from a real-time audio path. The detector never consults it; the timer performs the match after the fact.

13 · The audio signal path

The single most delicate piece of code in the project, and the one place where changing the order of two statements introduces a real bug.

The application requests one input channel and two output channels. When input channels are fewer than output channels, JUCE's `AudioSourcePlayer` copies the live input into the very buffer it is about to send to the output, *before* the callback runs. That single fact drives the whole design:

```
void MainComponent::getNextAudioBlock (const AudioSourceChannelInfo& bufferToFill)
{
    // 1. CAPTURE first - the buffer still holds the live guitar signal
    if (bufferToFill.buffer->getNumChannels() > 0)
        appendToCaptureBuffer (bufferToFill.buffer->getReadPointer (0, ...),
                                bufferToFill.numSamples);

    // 2. CLOBBER - never mix; overwriting kills the input passthrough
    bufferToFill.clearActiveBufferRegion();

    // 3. Only now is it safe to add the tone
    referenceTonePlayer.renderNextBlock (*bufferToFill.buffer, ...);
}
```

Capture, then clobber — never mix

Clearing the buffer does two jobs at once. It removes JUCE's automatic input passthrough, so the guitar does not leak to the speakers unbidden; and it guarantees the reference tone is written into an empty buffer that has already been read, so the tone can never be captured and fed to the pitch detector. Reordering these three steps, or mixing instead of clearing, reintroduces exactly the feedback loop the design exists to prevent.

Windowing and analysis rate

Constant	Value	Effect
<code>windowSize</code>	2048 samples	Analysis window. About 43 ms at 48 kHz — long enough to resolve low E (82.4 Hz) reliably.
<code>hopSize</code>	512 samples	Analysis runs once per hop, roughly 94 times a second at 48 kHz — far more often than the 30 Hz display refresh.
Timer rate	30 Hz	Display refresh. Independent of analysis rate; the UI simply reads the most recent result.

Input is accumulated into a circular buffer. Each hop, the most recent `windowSize` samples are unwrapped into a contiguous scratch buffer and analysed. Both buffers are allocated once in `prepareToPlay()`.

14 · Pitch detection — the YIN algorithm

`PitchDetector` implements YIN, an autocorrelation method well suited to monophonic instruments. It is hand-written rather than taken from a library, and depends on nothing but the standard library.

How it works

Step	What happens
1. Difference function	For each candidate lag τ , sum the squared difference between the signal and a copy of itself shifted by τ . Where the waveform repeats, this dips toward zero.
2. Cumulative mean normalisation	Divide each value by the running mean of all values up to that lag. Without this the function is trivially smallest at $\tau=0$ and the true period is never found.
3. Absolute threshold	Walk forward and take the first lag dropping below 0.15, then follow it down to its local minimum. Taking the first qualifying dip rather than the global minimum is what avoids octave errors.
4. Parabolic interpolation	Fit a parabola through the minimum and its two neighbours. Without this, pitch would be quantised to whole samples — far too coarse for cent-level accuracy.

Tuning parameters

Constant	Value	Rationale
<code>yinThreshold</code>	0.15	How periodic a signal must be to count as pitched. Lower is stricter.
<code>silenceRmsThreshold</code>	0.003	Roughly -50 dBFS. Below this the detector reports no pitch rather than analysing noise.
<code>minDetectableFrequencyHz</code>	70.0	Below open low E (82.4 Hz) with margin for a badly flat string.
<code>maxDetectableFrequencyHz</code>	1400.0	Above the highest fretted note on the top string.

The frequency bounds matter for performance as well as correctness: they are converted once in `prepare()` into a minimum and maximum lag, so the search scans a few hundred candidates instead of the whole window.

Confidence

`Result::confidence` is $1.0 - \text{cmnd}[\tau]$: how cleanly the waveform matched itself at the chosen lag. Because a result is only returned when the CMND value is already below 0.15, every reported confidence necessarily exceeds 0.85. The field is currently computed and published but not used for filtering — see §22.

15 · The plucked string synthesiser

`PluckedString` generates the reference tone by additive synthesis: ten sine partials at exact integer multiples of the fundamental, each with its own amplitude and its own decay rate.

Why not Karplus-Strong

Karplus-Strong is the classic plucked-string algorithm and would sound at least as convincing, but it sets pitch by the length of a delay line — quantised to whole samples. At guitar frequencies that lands measurably off the target, and a reference tone for a *tuner* must be exactly on pitch. Additive synthesis places every partial at an exact ratio by construction. The test suite verifies the fundamental measures 109.997 Hz against a 110 Hz target.

What makes it sound like a string

Property	Model	Perceptual effect
Harmonic amplitudes	$ \sin(k\pi p) / k^{1.2}$, with pluck position $p = 0.2$	The sin term reproduces the notch an ideal string has at harmonics that fall on the pluck point — here the fifth vanishes. The exponent is gentler than an ideal string's k^2 , which alone sounds dull.
Per-harmonic decay	Time constant $\tau/k^{0.7}$	Upper partials die faster than the fundamental, so the note is bright at the attack and mellows as it rings. This is the strongest cue that the ear reads as a real string.
Attack	4 ms ramp	Fast enough to sound percussive, slow enough not to click.
Normalisation	Amplitudes divided by their sum	Bounds the peak at 1.0 so the output level does not shift if the timbre is retuned.

Aliasing guard

Any partial landing at or above 45% of the sample rate is dropped rather than rendered. Without this, high reference pitches would fold their upper harmonics back down the spectrum as inharmonic tones — audible as a metallic ring.

A note declares itself finished when the fundamental's envelope falls below a small threshold; since it outlasts every other partial, it alone decides when the note is over. That flag is what the looping logic waits on.

16 · Reference tone playback

`ReferenceTonePlayer` wraps the synthesiser with the threading discipline the audio path demands, plus the retrigger and loop behaviour.

Public interface

Method	Thread	Effect
<code>prepare(sampleRate, blockSize)</code>	Setup	Sizes internal state. Allocates; must not be called from the audio thread.
<code>setFrequency(hz)</code>	Message	Takes effect on the next pluck, so retuning never bends a ringing note.
<code>pluck()</code>	Message	Sounds a note at the current frequency.
<code>setLooping(bool)</code>	Message	Re-plucks each time the previous note finishes decaying.
<code>renderNextBlock(...)</code>	Audio	Adds the note into the buffer. Allocation- and lock-free.

Retriggering without a click

Interrupting a ringing note is the one operation that can produce an audible click. A fresh pluck resets all phases to zero, so the waveform instantaneously becomes zero while the previous note was still at some non-zero amplitude — a step discontinuity.

The player therefore checks whether a note is sounding. If it is, the gain is ramped down over 8 ms and the pluck is deferred until that ramp completes; if nothing is sounding, the pluck happens immediately. Looping needs no such fade, because the previous note has by definition already decayed to silence.

Measured, not assumed

The test suite quantifies this. A click would be a sample-to-sample step comparable to the signal's own peak. A clean start and a mid-ring retrigger both measure a jump-to-peak ratio of 0.07 — identical, and far below the 0.30 bar that distinguishes harmonic slope from a discontinuity.

17 · The tuning model

PitchMath

Header-only free functions: MIDI note to frequency, frequency ratio to cents, MIDI note to name. Shared by the presets and the engine so the conversion exists in exactly one place.

Tuning and TuningPresets

Tuning is plain data: a name and six `TuningString` entries, low to high, each a note name and a frequency baked at A4 = 440 Hz. Presets are declared as MIDI note numbers, which keeps interval relationships legible and leaves name and frequency derivation in one helper. `TuningPresets::all()` returns them in display order.

TuningEngine

Owns the current tuning and the reference pitch, and answers one question: given a detected frequency, which string is it nearest and by how many cents? Nearest is measured in cents rather than hertz, which is the perceptually correct metric — a 5 Hz error at the low E is a much larger musical interval than the same 5 Hz at

the top string.

```
// Preset frequencies are baked at A4 = 440Hz, so the whole tuning
// scales with the reference pitch.
double TuningEngine::getStringFrequencyHz (int stringIndex) const
{
    return strings[stringIndex].frequencyHz * (referenceA4 / 440.0);
}
```

18 · The user interface layer

A small design system sits under the components, so colour, spacing and type are defined once rather than scattered through paint methods.

File	Responsibility
Theme.h	Design tokens — palette, corner radii, spacing scale, tuning tolerances, font helpers. Also owns <code>tuneColour()</code> , the single place cents maps to a feedback colour.
GlassStyle	Three painting primitives: the aurora backdrop, a translucent glass surface, and a coloured glow. Free functions, so any component can opt in from its own <code>paint()</code> .
TunerLookAndFeel	Applies the language to stock JUCE controls. Installed as the default look and feel, so the options dialog inherits it without being restyled by hand.
TunerDisplayComponent	The main readout. Computes its layout once in a helper shared by <code>paint()</code> and <code>resized()</code> , so the two can never disagree.
NeedleMeterComponent	The arc gauge. Owns a 60 Hz timer and eases toward each new reading, repainting only while something is actually moving.
ReferenceToneBar	The six string buttons and the Loop toggle. A pure control surface: it reports intent and reflects state, and owns no audio.
OptionsPanel	Contents of the Options dialog — the A4 slider and JUCE's device selector.

Backdrop caching

The glass card is translucent, so every repaint of the display forces the backdrop beneath it to repaint too — thirty times a second, including three radial gradients. The backdrop is therefore rendered once into an off-screen image and blitted thereafter, and invalidated only on resize.

19 · Settings persistence

`TunerSettings` wraps `juce::ApplicationProperties`. Writes are marked dirty and flushed on a two-second delay, so dragging the A4 slider does not touch the disk on every value change.

Key	Type	Restored by
<code>audioDeviceState</code>	XML	Passed into <code>setAudioChannels()</code> as stored settings, so the device opens correctly on the first frame rather than being switched afterwards.
<code>tuningName</code>	String	Matched against preset names at startup. Stored by name, not index, so reordering presets cannot silently restore the wrong tuning.

`referencePitchHz` Double Applied to both the engine and the slider, the latter without firing its change callback.

Device state is written on every device change as well as at shutdown, so an unexpected termination cannot lose the selection.

20 · Tests and developer tools

TunerTests

A console application with no third-party test framework — assertions print a PASS/FAIL line and the process exits non-zero if anything failed. It deliberately includes a self-check that a wrong expectation is reported as a failure, so a silently broken harness cannot masquerade as a passing suite.

Test	Asserts
Open string detection	All six standard-tuning frequencies detected within a cent from synthetic sines.
Silence	No pitch reported from a silent buffer.
Plucked tone pitch	The reference tone measures 109.997 Hz against a 110 Hz target — verified by feeding the synthesiser's output back through the detector.
Harmonic richness	Crest factor exceeds a pure sine's, confirming real harmonic content.
Decay	A one-shot note stops ringing without intervention.
Click-free start	Largest sample-to-sample jump stays far below the peak amplitude.
Click-free retrigger	Interrupting a ringing note is as clean as a fresh start.
Loop behaviour	Looping still sounds after several note lengths; one-shot is silent.

DesignPreview

A developer tool that renders the tuner display in four tuning states — in tune, slightly sharp, very flat, no signal — plus the options panel, straight to PNG. It exists because UI work otherwise requires a guitar in hand to see anything but the idle state, and because a rebuild-and-squint loop is a slow way to judge layout. It found two real bugs during development: a needle drawn detached from its pivot, and a stale reading held while the display read *no signal*.

21 · File reference

File	Lines of interest
Source/Main.cpp	JUCEApplication, window creation, installs TunerLookAndFeel as the default.
Source/MainComponent.*	Audio callback, thread handover, layout, wiring between every other class.
Source/PitchDetector.*	YIN implementation.
Source/PitchMath.h	MIDI, frequency, cents and note-name conversions.
Source/Tuning.*	Tuning data type.
Source/TuningPresets.*	The seven presets, declared as MIDI note numbers.
Source/TuningEngine.*	Nearest-string matching and reference-pitch scaling.
Source/PluckedString.*	Additive plucked-string synthesis.
Source/ReferenceTonePlayer.*	Thread-safe wrapper, retrigger fade, looping.
Source/ReferenceToneBar.*	String buttons and Loop toggle.

File	Lines of interest
Source/TunerDisplayComponent.*	Note name, cents, frequency, hint; hosts the meter.
Source/NeedleMeterComponent.*	Arc gauge with easing animation.
Source/OptionsPanel.*	Options dialog contents.
Source/TunerSettings.*	Persistence.
Source/Theme.h	Design tokens.
Source/GlassStyle.*	Backdrop, glass surface and glow painting.
Source/TunerLookAndFeel.*	Styling for stock JUCE controls.
Tests/TunerTests.cpp	The test suite.
Tools/DesignPreview.cpp	PNG preview renderer.

22 · Known limitations and deferred scope

Known limitations

Area	Detail
Unused confidence value	PitchDetector publishes a confidence score that nothing reads. It was evaluated as a noise filter and set aside: with a properly gained instrument input the detector is reliable without it, and it cannot reject the case it was intended for — mains hum harmonics are genuinely periodic and score highly. Left in place because it is meaningful diagnostic data.
Nearest-string matching	A string more than roughly half a semitone out can match its neighbour and display the wrong name. A mode locking the tuner to one chosen string would resolve this.
Noise on weak inputs	With a laptop microphone rather than an interface, room noise and mains hum can register as notes. The RMS gate is set low (−50 dBFS) to avoid missing quiet playing.
Out-of-range cents	Offsets beyond the meter's ± 50 cent span are displayed numerically even though the needle has reached its limit.

Deferred by design

The following were considered and consciously left out of this version, not overlooked:

Feature	Status
Custom tuning builder	Specified but unbuilt. The seven presets cover common use; per-string configuration remains open.
VST3 / AAX plugin formats	Standalone only. The DSP layer is deliberately structured so this could be added without rework; AAX additionally requires an Avid developer agreement and PACE signing.
Stability / hold before confirming	Requiring several consecutive agreeing readings would suppress flicker, at the cost of responsiveness. Excluded from this version's scope.
Strobe tuner display	An alternative to the needle. Not implemented.
Bass guitar range	Detection floor is 70 Hz; a low B at ~31 Hz sits well below it.
Chord / polyphonic detection	Explicitly out of scope. The algorithm is monophonic by construction.

A note on the analysis window

The original specification left the analysis window size open, to be settled by measurement. It has been running at 2048 samples with a 512-sample hop throughout development and testing without complaint, and that pairing is now treated as settled. Shortening the window would improve responsiveness but degrade accuracy on the low E, where a single period alone occupies close to 600 samples.